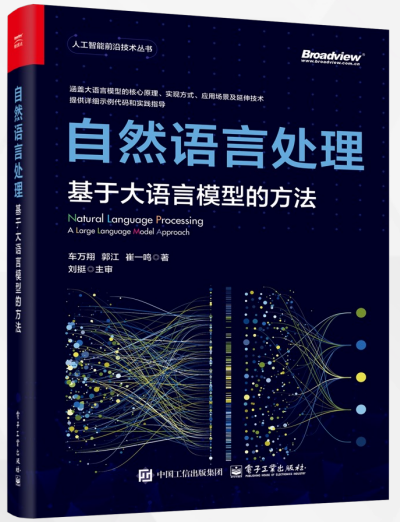


语言模型的微调

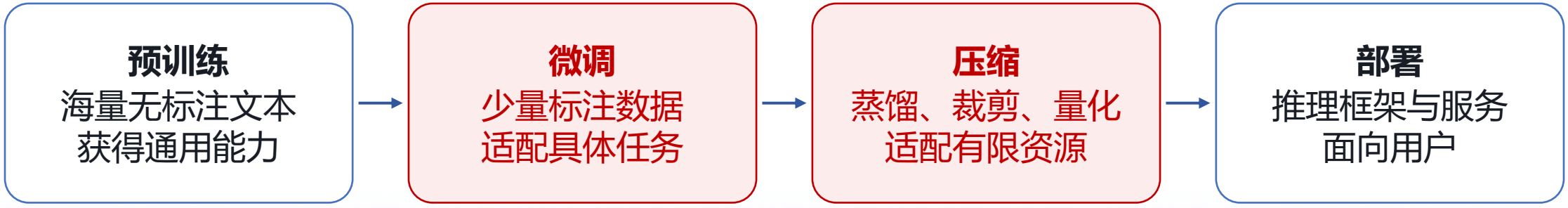
车万翔

社会计算与交互机器人研究中心（HIT-SCIR）
哈尔滨工业大学

本讲内容与学习目标

课堂内容	学习目标
微调的基本方法	说清微调做什么、怎样设置优化器与学习率
微调的实践要点	应对遗忘、不稳定与显存不足
参数高效精调	会用LoRA与QLoRA，能计算可训练参数量
生成任务的微调	正确构造标签与损失掩码
知识蒸馏	理解软标签、温度与DistilBERT
模型裁剪	区分非结构化、结构化与半结构化裁剪
参数量化与练习	掌握Absmax、LLM.int8()与GPTQ的思想

从预训练到部署



效果问题 怎样在少量标注数据上把预训练模型用好，而不忘掉已学的知识?

成本问题 怎样在几乎不损失效果的前提下，让模型更小、更快、更省显存?

提示学习、指令微调、人类反馈强化学习和参数高效精调见下一讲“大语言模型的适配”。

微调在做什么

$$\theta^* = \arg\min_\theta \sum_{(x,y)\in\mathcal{D}} \mathcal{L}(f(x;\theta), y), \quad \text{初始值 } \theta = \theta_0 \text{（预训练参数）}$$

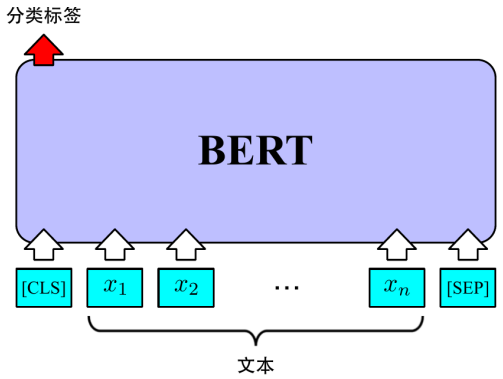
对比	预训练	微调
数据	海量无标注文本	几千到几十万条标注样本
目标	自监督（语言模型、MLM等）	任务损失（分类、标注、生成）
学习率与轮数	较大，训练很多步	小一到两个数量级，通常2～4轮
参数	全部随机初始化	主体来自θ_0，只新增少量任务层

为什么有效　　θ_0已经包含语言知识，任务只需在其附近做小幅调整

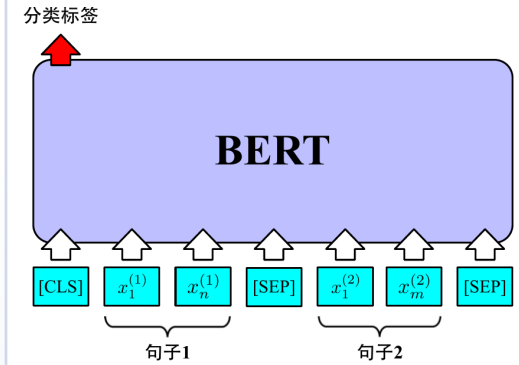
四种使用预训练模型的方式

方式	更新哪些参数	每个任务需保存	典型场景
特征提取	只训练任务层	任务层	资源受限、快速基线
全参数微调	全部参数	一整份模型	数据充足、追求效果
参数高效精调	少量新增参数	小型增量参数	大模型、多任务
提示 / 语境学习	不更新参数	提示模板与示例	大模型零样本、小样本

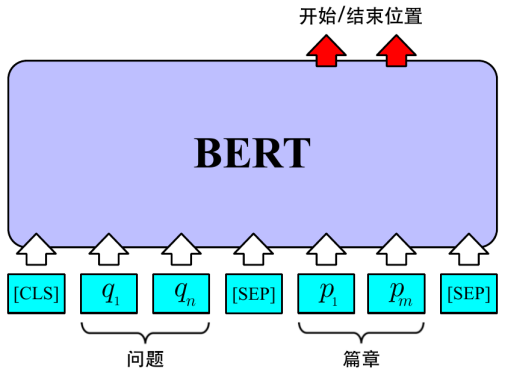
回顾：四类任务的微调结构



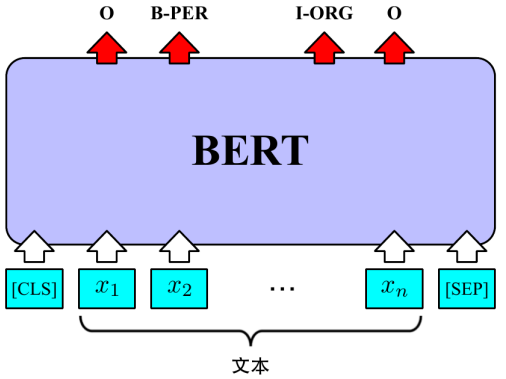
单句分类：h_0 → K类



句对分类：h_0 → K类



阅读理解：起止位置



序列标注：每位置标签

同一个预训练模型：只换输入拼接方式与输出层。

输入格式与输出层

任务	输入序列	输出层	损失
单句分类	[CLS] x [SEP]	h_0 → K类Softmax	交叉熵
句对分类	[CLS] $x^{(1)}$ [SEP] $x^{(2)}$ [SEP]	h_0 → K类Softmax	交叉熵
阅读理解	[CLS] 问题 [SEP] 篇章 [SEP]	每个位置 → 起点、终点分数	$(\mathcal{L}^s + \mathcal{L}^e) / 2$
序列标注	[CLS] x [SEP]	每个位置 → K个BIO标签	逐位置交叉熵

$$P = \mathrm{Softmax}(h_0 W^o + b^o), \quad W^o \in \mathbb{R}^{d \times K}$$

抽取式阅读理解：预测起止位置

【篇章】
……学校于1996年进入国家“211工程”首批重点建设高校，**1999年**被确定为国家首批“985工程”重点建设的9所大学之一……
【问题】
哈尔滨工业大学在哪一年入选了国家首批“985工程”？
【答案】1999年

$$P^s = \text{Softmax}(h W^s + b^s)$$

$$P^e = \text{Softmax}(h W^e + b^e)$$

$$\mathcal{L} = (\mathcal{L}^s + \mathcal{L}^e) / 2$$

解码：起点、终点各取Top-k，
按$P^s_i \cdot P^e_j$从大到小，
选第一个满足“起点 ≤ 终点”的片段。

输入顺序　　问题在前、篇章在后：超长时只截断篇章，可滑窗切片多次预测

数据集：SQuAD（英文）、CMRC 2018（中文）。答案是篇章中的一个连续片段。

序列标注：子词与标签对齐

	John	Smith	has	never	been	to	Harbin	
原始标签	B-PER	I-PER	O	O	O	O	B-LOC	
切分后	John	Smith	has	never	been	to	**Ha**	**##rbin**
对齐后标签	B-PER	I-PER	O	O	O	O	**B-LOC**	**B-LOC**

BIO B：实体开始；I：实体内部（Inside）；O：非实体

对齐策略 教材：子词继承原标签；常见做法：只在首个子词计算损失，其余设为−100

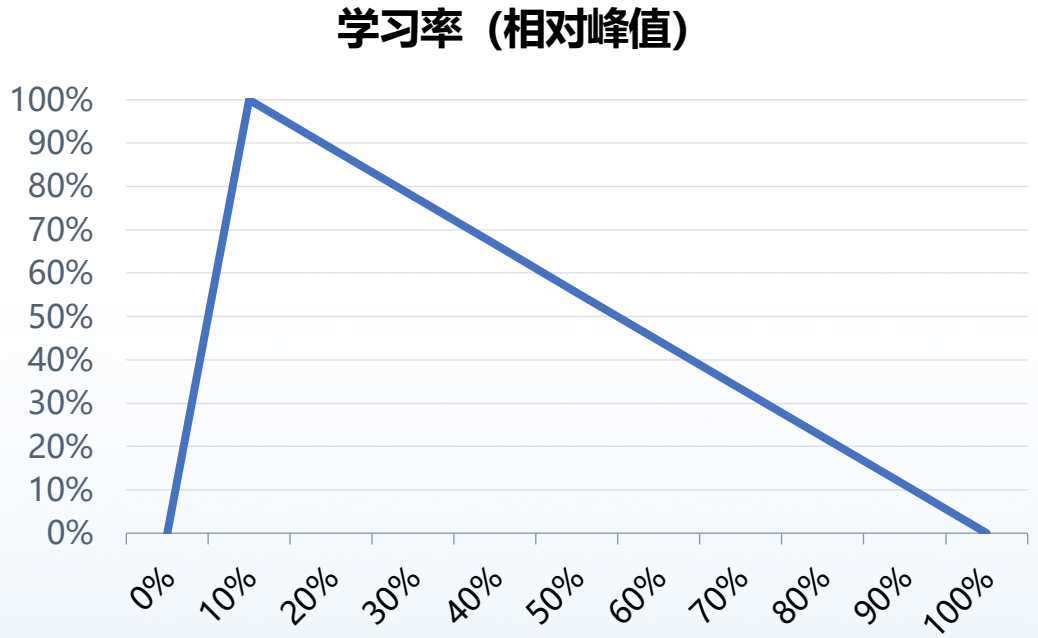
代码实现：用transformers精调BERT

```
from datasets import load_dataset
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          TrainingArguments, Trainer)
import evaluate, numpy as np

data = load_dataset("nyu-mll/glue", "sst2")
tok = AutoTokenizer.from_pretrained("bert-base-cased")
data = data.map(lambda b: tok(b["sentence"], truncation=True), batched=True)
model = AutoModelForSequenceClassification.from_pretrained("bert-base-cased", num_labels=2)
acc = evaluate.load("accuracy")
def metrics(p):
    return acc.compute(predictions=np.argmax(p.predictions, -1), references=p.label_ids)
args = TrainingArguments("ft-sst2", eval_strategy="epoch", learning_rate=2e-5,
                         per_device_train_batch_size=16, num_train_epochs=2)
trainer = Trainer(model=model, args=args, train_dataset=data["train"],
                  eval_dataset=data["validation"], processing_class=tok,
                  compute_metrics=metrics)
trainer.train()
```

微调的优化设置

超参数	BERT论文的搜索范围
批次大小	16，32
学习率	5e-5，3e-5，2e-5
训练轮数	2，3，4
预热比例	前10%的步数
权重衰减	0.01（偏置、LayerNorm除外）



横轴：已训练步数的比例。前10%预热，之后线性衰减到0。

经验 在验证集上选超参数；小数据集更要多试几个学习率

PyTorch实现：AdamW与预热线性衰减

```
from torch.optim import AdamW
from transformers import get_linear_schedule_with_warmup

no_decay = ("bias", "LayerNorm.weight")              # 不做权重衰减的参数
groups = [
    {"params": [p for n, p in model.named_parameters() if not n.endswith(no_decay)],
     "weight_decay": 0.01},
    {"params": [p for n, p in model.named_parameters() if n.endswith(no_decay)],
     "weight_decay": 0.0}]
opt = AdamW(groups, lr=2e-5)
total = len(loader) * epochs
sched = get_linear_schedule_with_warmup(opt, int(0.1 * total), total)   # 10%预热
for epoch in range(epochs):
    for batch in loader:
        loss = model(**batch).loss
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)          # 梯度裁剪
        opt.step(); sched.step(); opt.zero_grad()
```

灾难性遗忘

$$\mathcal{L}(\mathcal{C}) = \mathcal{L}^{\mathrm{FT}}(\mathcal{C}) + \lambda\, \mathcal{L}^{\mathrm{PT}}(\mathcal{C})$$

教材第7章：GPT精调时加入语言模型损失，GPT论文取$\lambda = 0.5$

做法	思路	代价
小学习率、少轮数	不把参数推离θ_0太远	可能欠拟合
加入预训练损失	同时优化任务与语言模型目标	每步多一次前向
回放通用数据	混入少量预训练或通用指令数据	需要额外数据
冻结主干	只训练少量新增参数	效果上限可能略低

遗忘在指令微调大模型时同样存在：只在窄领域数据上训练，通用能力可能下降。

生成任务的微调：标签与损失掩码

输入

用户：	把	cheese	译成	法语	助手：	fromage	<eos>	<pad>	<pad>

标签

−100	−100	−100	−100	−100	−100	**fromage**	**<eos>**	−100	−100

只算回答　　提示与填充位置的标签设为−100，交叉熵会忽略它们

右移一位　　标签与输入相同，模型内部用第t个位置的输出预测第t+1个词元

常见问题　　填充位置未掩码：浪费计算，稀释回答部分的训练信号

灰色：提示（不计损失）；红色：回答（计损失）；白色：填充（不计损失）。

PyTorch实现：只在回答部分计算损失

```
prompt = tok("User: Translate to French: cheese\nAssistant:")["input_ids"]
answer = tok(" fromage")["input_ids"] + [tok.eos_token_id]
input_ids = torch.tensor([prompt + answer])
labels = input_ids.clone()
labels[0, :len(prompt)] = -100                          # 提示部分不计损失
out = model(input_ids=input_ids, labels=labels)         # 模型内部把标签右移一位

# 等价的手工计算：用第t个位置的输出预测第t+1个词元
logits, target = out.logits[:, :-1], labels[:, 1:]
manual = F.cross_entropy(logits.reshape(-1, logits.size(-1)),
                         target.reshape(-1), ignore_index=-100)
assert torch.allclose(out.loss, manual)
```

生成任务的微调：序列到序列

环节	做法
编码器输入	任务前缀 + 源文本，如 translate English to German: That is good
标签	目标文本的词元序列，末尾加结束符，填充位置设为−100
解码器输入	标签右移一位，开头放起始符（T5用<pad>作起始符）
评价	翻译：BLEU、chrF；摘要：ROUGE；同时人工检查样例

对比

GPT类：条件与目标拼成一条序列；T5类：编码器读条件，解码器生成目标

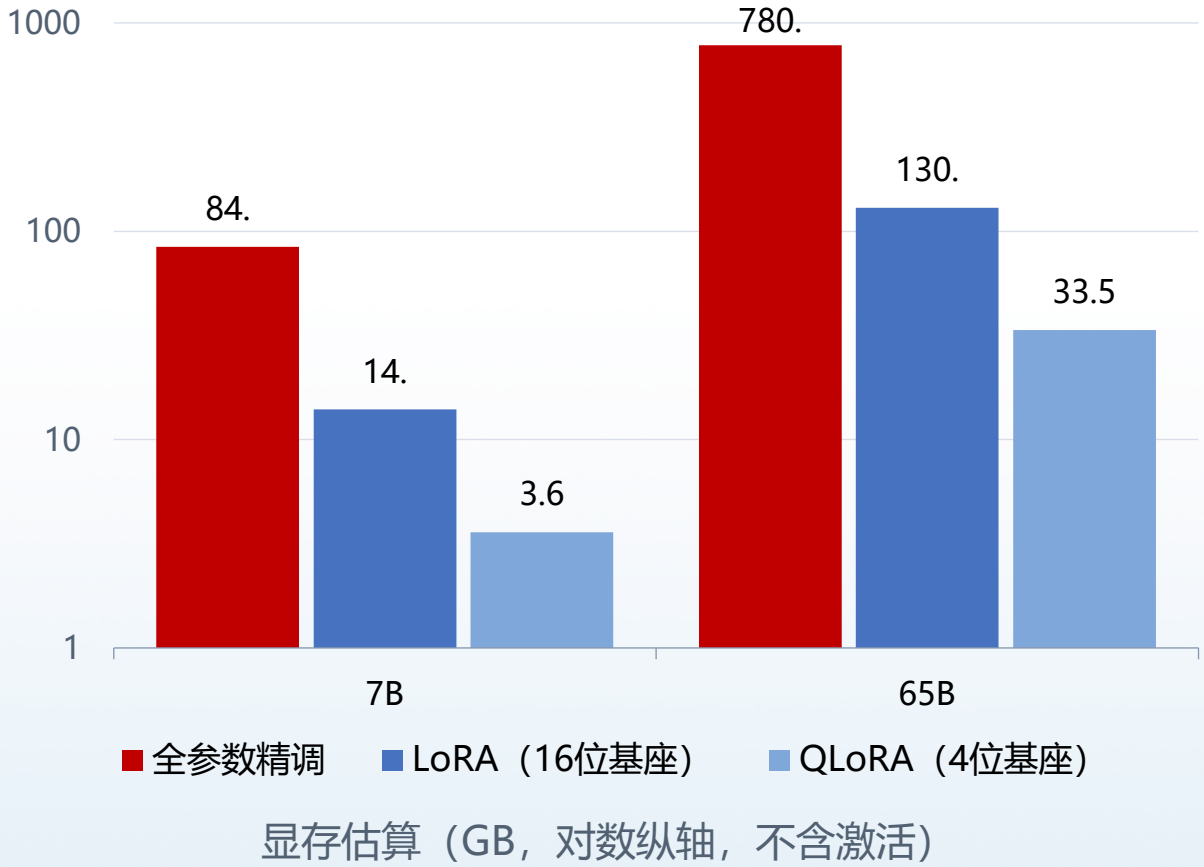
微调的不稳定性

现象	同样的超参数，只换随机种子，结果可能差很多（小数据集上尤甚）
来源	任务层的初始化与数据顺序两者影响相当（Dodge等，2020）
原因	主要是优化困难：训练早期梯度消失（Mosbach等，ICLR 2021）
对策	Adam偏差校正、小学习率、更多训练步；提前停止明显失败的运行

报告结果：至少5个随机种子的均值 ± 标准差，并在验证集上选模型。

BERT论文：BERT-large在小数据集上微调有时不稳定，作者多次随机重启后按验证集选优。

为什么需要参数高效精调



显存估算（GB，对数纵轴，不含激活）

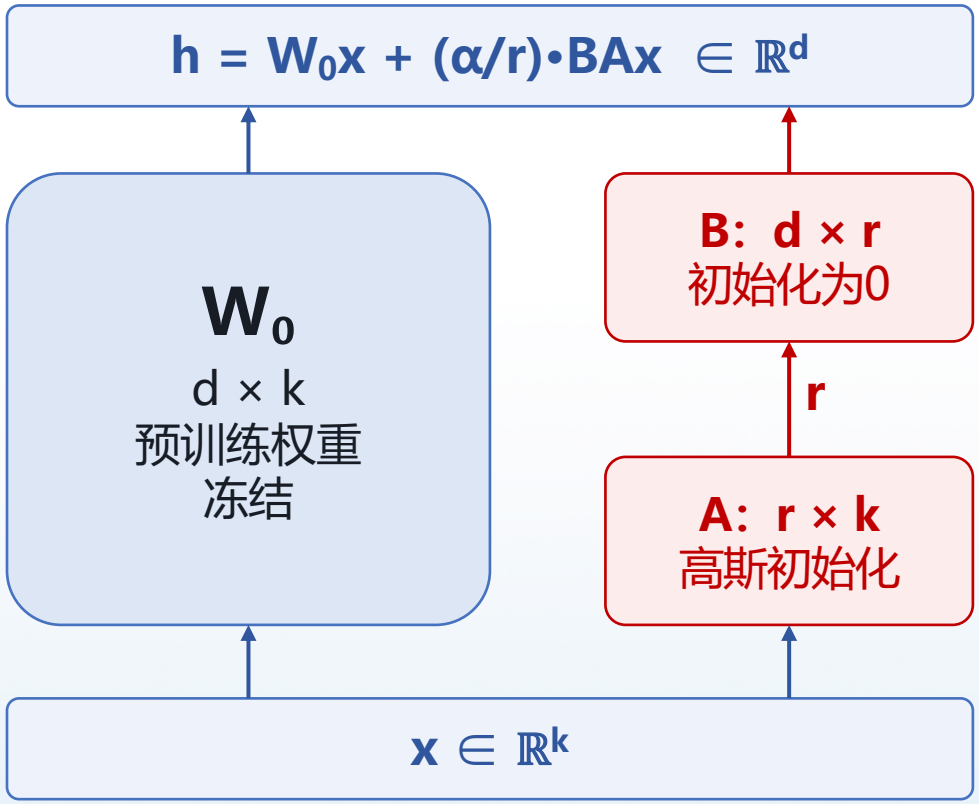
加入新模块 Adapter：在层间插入小型瓶颈网络

软提示 Prefix-tuning、P-tuning、Prompt-tuning

低秩重参数化 LoRA、QLoRA：用两个小矩阵表示权重增量

全参数按约12字节/参数估算（与QLoRA论文“65B需780GB以上”一致）；NF4含双重量化约4.127比特/参数。

LoRA：低秩适配



假设 微调时的权重变化ΔW是低秩的

初始化 B = 0，A为高斯分布：开始时ΔW = 0

缩放 α/r；确定r后通过α调节

参数量 d=4096，k=11008，r=64：4508.9万 → 96.7万（2.1%）

推理 合并$W = W_0 + (\alpha/r)BA$，无额外延迟

维度按LoRA论文：x为k维输入、h为d维输出（教材注释写反了）。Hu等，LoRA，ICLR 2022。

用peft实现LoRA

```
from transformers import AutoModelForCausalLM
from peft import LoraConfig, TaskType, get_peft_model, PeftModel

config = LoraConfig(task_type=TaskType.CAUSAL_LM,
                    target_modules=["q_proj", "v_proj"],   # 在哪些权重上加LoRA
                    r=8, lora_alpha=32, lora_dropout=0.1)
model = AutoModelForCausalLM.from_pretrained("hfl/chinese-llama-2-7b")
model = get_peft_model(model, config)
model.print_trainable_parameters()
# trainable params: 4,194,304 || all params: 6,933,450,752 || trainable%: 0.0605
...                                          # 用Trainer训练
model.save_pretrained("output_dir")          # 只保存LoRA权重（约17MB）

base = AutoModelForCausalLM.from_pretrained("hfl/chinese-llama-2-7b")
model = PeftModel.from_pretrained(base, "output_dir")   # 基座 + LoRA
model = model.merge_and_unload()             # 可选：合并后推理无额外开销
```

实测：peft 0.21，transformers 5.19。教材中的输出为占位数字，实测为0.0605%。

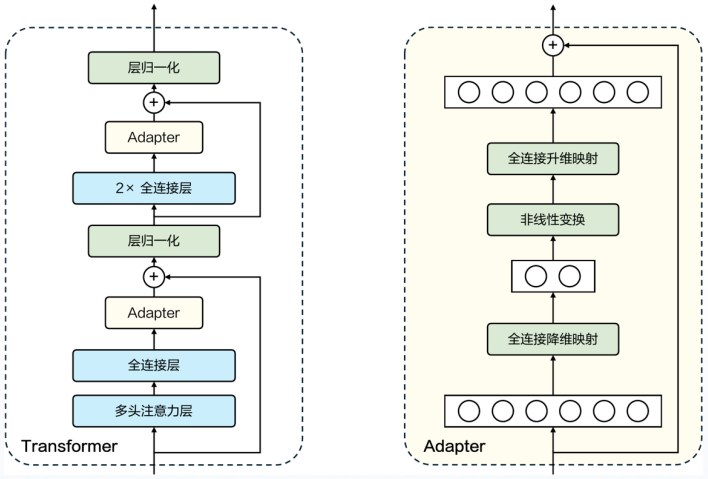
LoRA的实践要点

问题	建议	依据
加在哪些层	所有线性层，尤其是MLP；只加注意力层明显更差	QLoRA论文；Schulman等，2025
秩r	常用8～64；数据量大时增大r；强化学习r很小即可	教材；Schulman等，2025
学习率	约为全参数微调的10倍	Schulman等，2025
遗忘	LoRA学得少，也忘得少	Biderman等，TMLR 2024
部署	合并权重；或一个基座同时服务多个LoRA	S-LoRA、vLLM

实测（Chinese-LLaMA-2-7B，r=64，所有线性层）：1.60亿可训练参数（2.26%）。

其他方法：Adapter与软提示

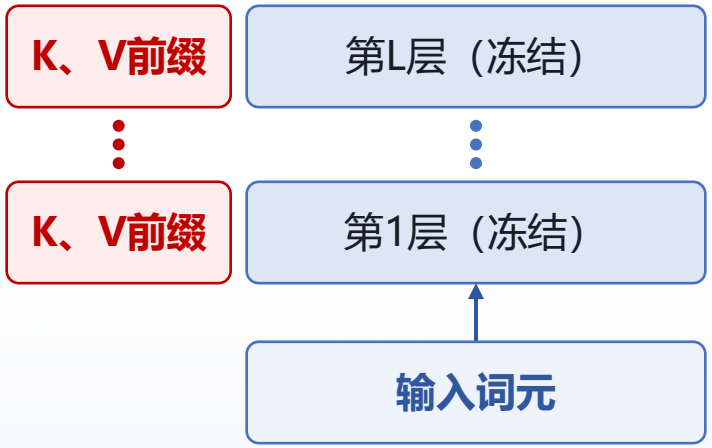
Adapter



参数量 = 2md + d + m

降维 → 非线性 → 升维 → 残差
升维层为0时是恒等映射

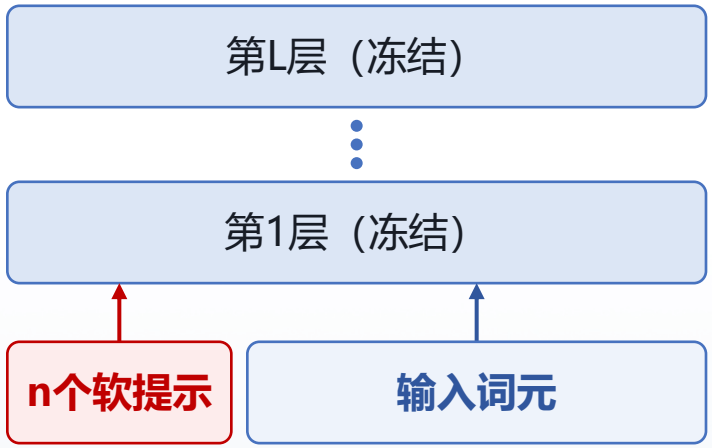
Prefix-tuning



参数量 = 2Lnd

L层，每层K、V前各加n个d维向量
用MLP重参数化，训练更稳定

Prompt-tuning



参数量 = nd

只在输入层前加n个d维向量
约100亿参数时与全参数微调相当

软提示：用可训练的“虚拟词元”代替人工写的提示，模型参数全部冻结

Houlsby等，2019；Li与Liang，2021；Liu等，2021；Lester等，2021。左图为教材图9.15。

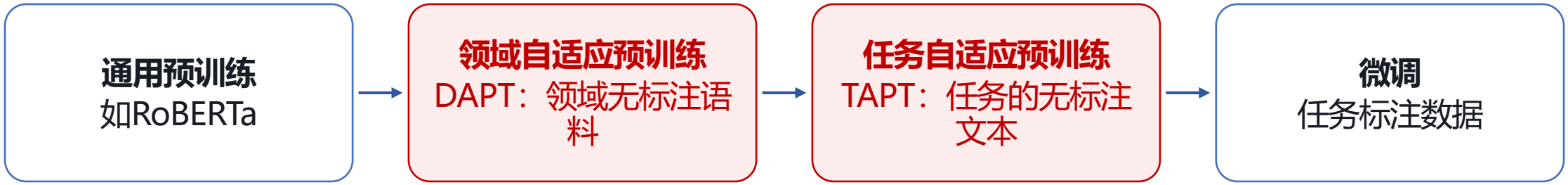
参数高效精调方法对比

方法（Llama-2-7B）	改动位置	可训练参数	推理额外开销	效果（原论文）
全参数微调	全部权重	67亿（100%）	无	基准
LoRA（q、v，r=8）	q、v的低秩增量	419万（0.06%）	合并后无	与全参数相当
QLoRA	同LoRA	同LoRA，基座为4位	解量化	与16位微调相当
Adapter（m=64）	每层插入瓶颈模块	3382万（0.50%）	多出的层	GLUE上比全参数低0.4
Prefix-tuning（长10）	每层K、V前的前缀	262万（0.04%）	序列变长	生成任务接近全参数 小数据时更好
Prompt-tuning（长100）	输入层前的软提示	41万（0.006%）	序列变长	约100亿参数时 与全参数相当

新进展：DoRA（2024）只对权重的方向做低秩更新

可训练参数为peft 0.21在Llama-2-7B上的实测值（Adapter按公式计算）；效果取自各原论文，模型与任务不同。
Hu等，2022；Dettmers等，2023；Houlsby等，2019；Li与Liang，2021；Lester等，2021；Liu等，2024。

继续预训练：领域与任务自适应



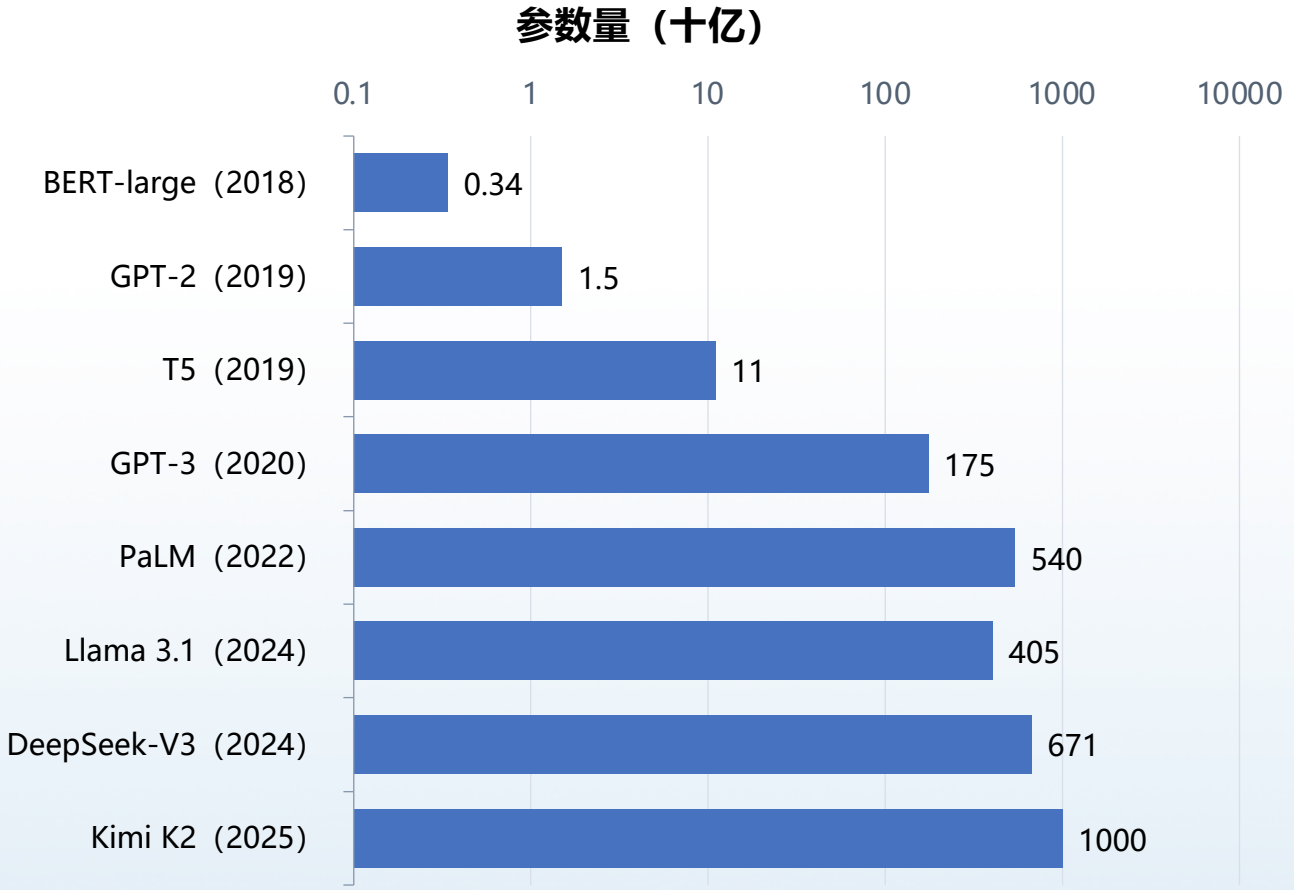
实验 4个领域（生物医学、计算机、新闻、评论）、8个分类任务

结论 DAPT与TAPT都优于直接微调，两者结合最好；目标领域与预训练语料差异越大，收益越明显

相关 中文增量训练：在中文语料上继续预训练英文大模型（第9章）

Gururangan等，Don’t Stop Pretraining，ACL 2020。

为什么需要模型压缩



7B模型权重	显存
FP32	28 GB
BF16	14 GB
INT8	7 GB
INT4	**3.5 GB**

蒸馏： 换一个小模型

裁剪： 去掉一部分参数

量化： 降低每个参数的精度

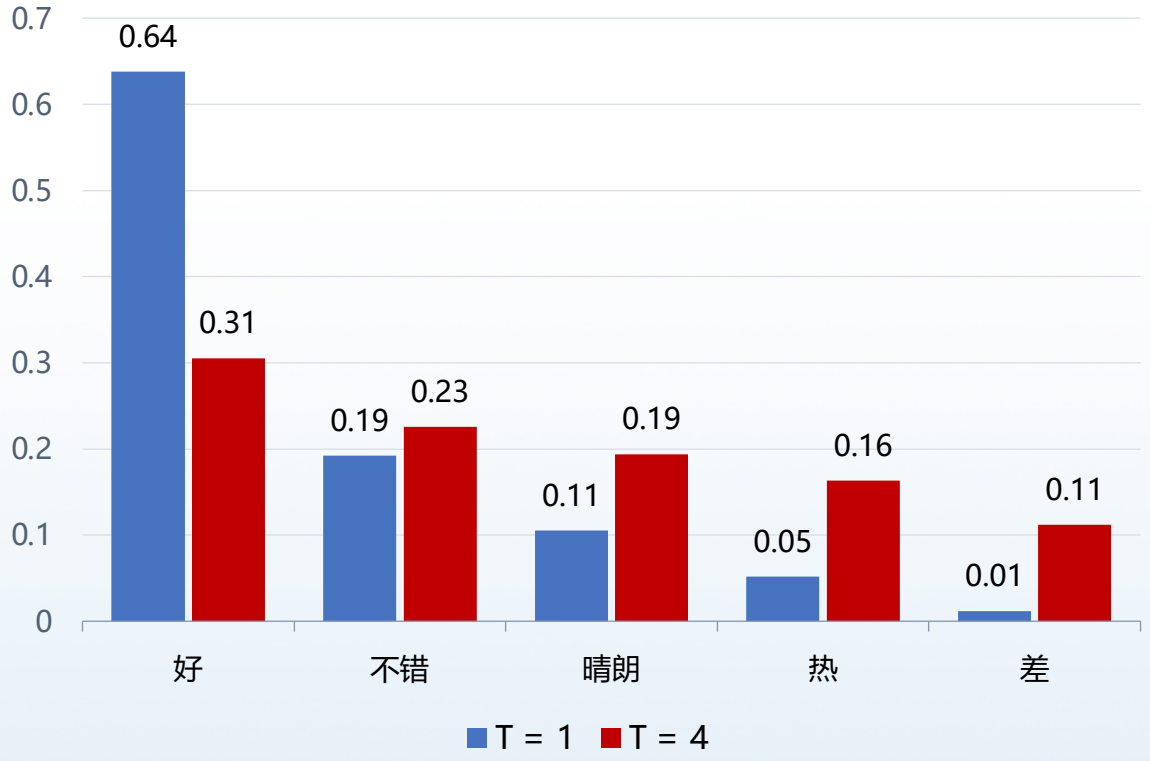
三条压缩路线

方法	改变了什么	是否需要训练	本讲示例
知识蒸馏	换成更小的学生模型	需要训练学生	DistilBERT、TextBrewer
模型裁剪	去掉不重要的权重或结构	视方法而定	TextPruner、SparseGPT、Wanda
参数量化	降低权重和激活的位宽	训练后量化无需训练	LLM.int8()、GPTQ、AWQ

三者可以叠加：先蒸馏或裁剪得到小模型，再量化部署。

知识蒸馏：软标签与温度

教师对“今天天气很[MASK]”的预测



$$p_i = \exp(z_i / T) / \Sigma_j \exp(z_j / T)$$

硬标签 只告诉学生“正确答案是好”

软标签 还告诉学生“不错、晴朗”与“好”更接近

温度T T越大分布越平，次要类别的信息越明显

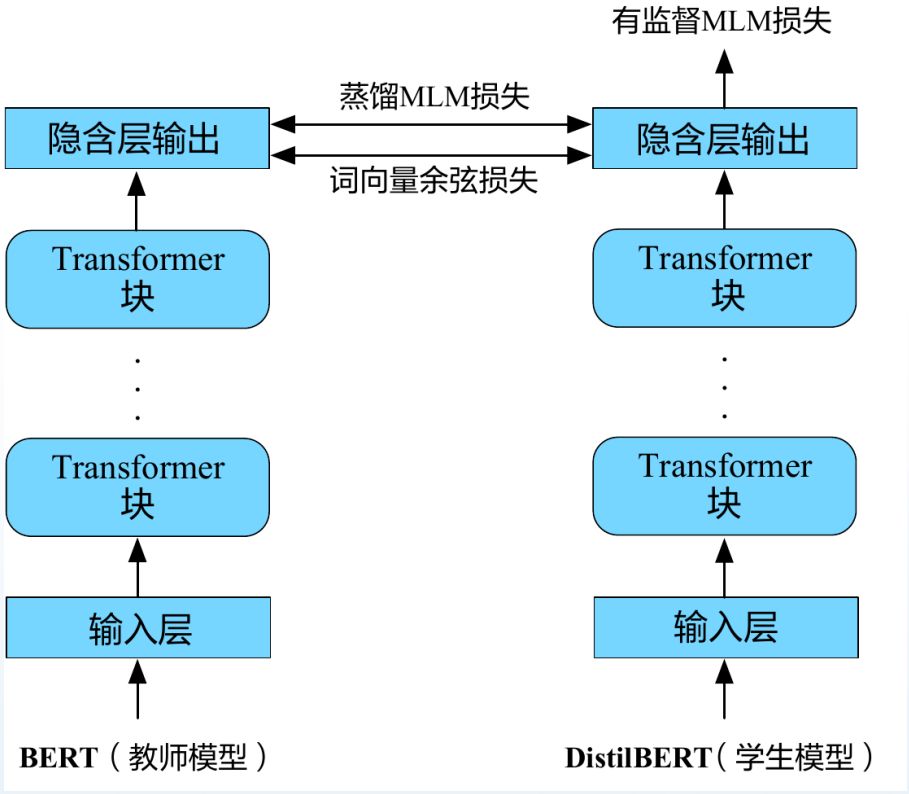
蒸馏损失：硬标签 + 软标签

$$\mathcal{L} = \alpha \cdot CE(y, s) + (1 - \alpha) \cdot T^2 \cdot KL(p_t^{(T)} \,||\, p_s^{(T)})$$

```
def kd_loss(s_logits, t_logits, labels, T=2.0, alpha=0.5):
    hard = F.cross_entropy(s_logits, labels)                    # 硬标签
    soft = F.kl_div(F.log_softmax(s_logits / T, dim=-1),        # 学生：对数概率
                    F.softmax(t_logits / T, dim=-1),            # 教师：概率
                    reduction="batchmean") * T * T               # 乘T²保持梯度量级
    return alpha * hard + (1 - alpha) * soft
```

软标签梯度范数	T = 1	T = 2	T = 4	T = 8
不乘T^2	0.095	0.032	0.009	0.002
乘T^2	0.095	0.126	0.146	0.150

DistilBERT：通用的小型BERT



项目	BERT-base	DistilBERT
层数	12	6
参数量	1.1亿	6600万
GLUE宏平均	79.5	**77.0（97%）**
CPU推理时间*	668秒	**410秒**
SQuAD 1.1 F1	88.5	85.8

初始化 从教师的12层中隔层取一层

训练 8块V100约90小时；动态掩码，不用NSP

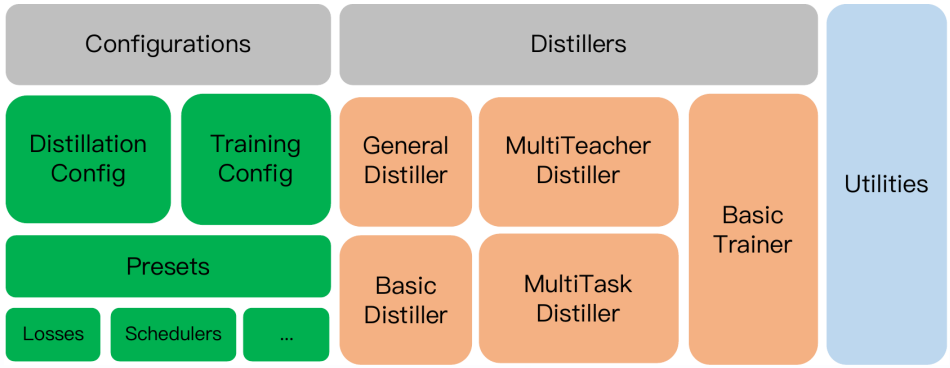
DistilBERT的三重损失

$$\mathcal{L} = \mathcal{L}_{ce} + \mathcal{L}_{mlm} + \mathcal{L}_{cos}$$

损失	含义
蒸馏损失 $\mathcal{L}ce$	$\Sigma\, t_i \log s_i$ 的负值：学生拟合教师在掩码位置的软标签（带温度T）
MLM损失 $\mathcal{L}mlm$	学生在掩码位置预测原词（硬标签）
余弦损失 $\mathcal{L}cos$	$1 - \cos(h^t, h^s)$：对齐学生与教师最后一层隐状态的方向

消融实验（相对完整模型）	GLUE宏平均变化
去掉蒸馏损失 $\mathcal{L}ce$	−2.96
去掉余弦损失 $\mathcal{L}cos$	−1.46
去掉MLM损失 $\mathcal{L}mlm$	−0.31
改为随机初始化学生	**−3.69**

TextBrewer：知识蒸馏工具包



第1步 在有标签数据上训练好教师模型

第2步 初始化学生，准备数据、优化器

第3步 定义配置与adaptor，调用train()

```
# adaptor：从模型输出中取出logits与硬标签损失
def adaptor(batch, out):
    return {"logits": out.logits, "losses": [out.loss]}

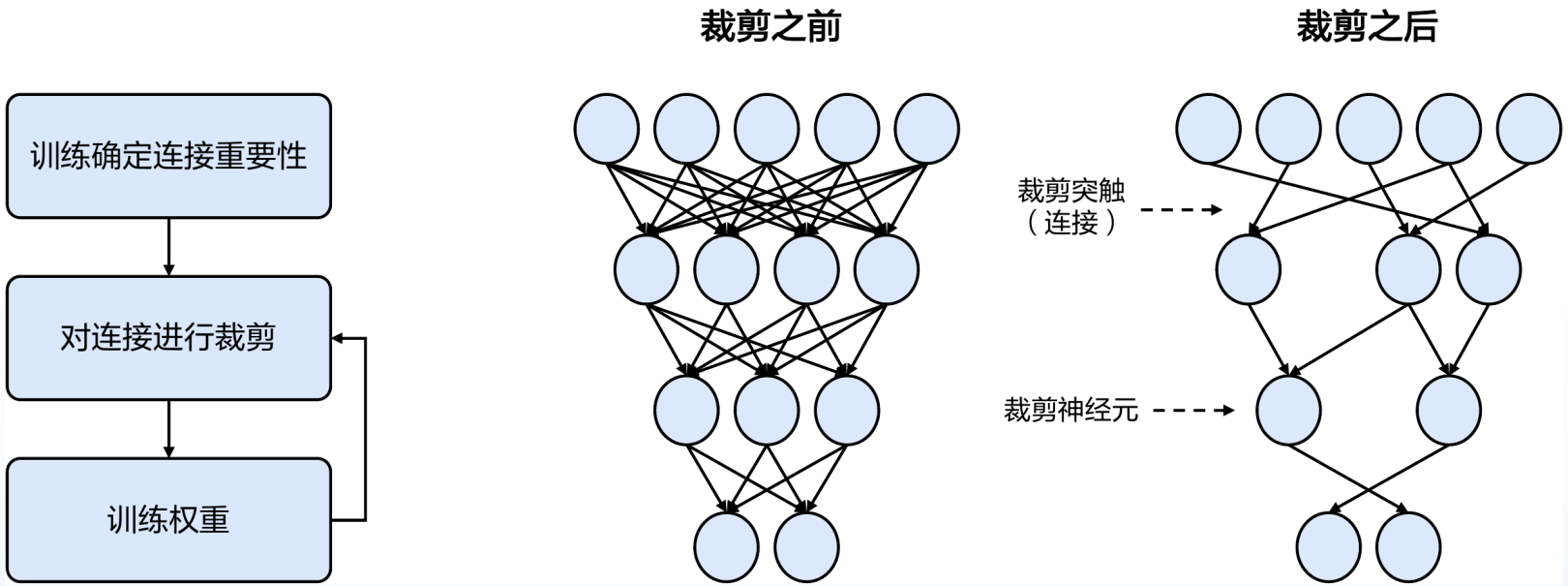
distiller = GeneralDistiller(
    train_config=TrainingConfig(device="cuda"),
    distill_config=DistillationConfig(
        temperature=4, kd_loss_weight=0.5,
        hard_label_weight=0.5),
    model_T=teacher, model_S=student,
    adaptor_T=adaptor, adaptor_S=adaptor)
with distiller:
    distiller.train(optimizer, loader, num_epochs=3)
```

大模型时代的蒸馏

方式	学生学什么	代表工作
黑盒（序列级）	教师生成的回答，直接监督微调	DeepSeek-R1蒸馏版：约80万条样本，1.5B～70B
白盒（词元级）	教师每一步的概率分布	MiniLLM：改用反向KL（ICLR 2024）
在策略蒸馏	学生自己生成，教师给出分布	GKD（ICLR 2024）；Qwen3小模型
预训练蒸馏	预训练时拟合教师分布	Gemma 2的2B、9B模型

趋势：小模型越来越多地由大模型“教”出来，而不是从零训练。

模型裁剪：去掉不重要的部分

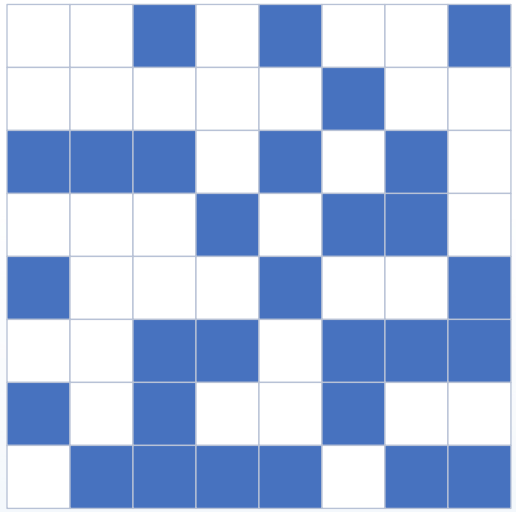


三个步骤　　重要性计算 → 按阈值裁剪 → 微调恢复性能（可多轮迭代）

幅值裁剪　　按权重绝对值排序，剪去最小的一部分（Han等，2015）

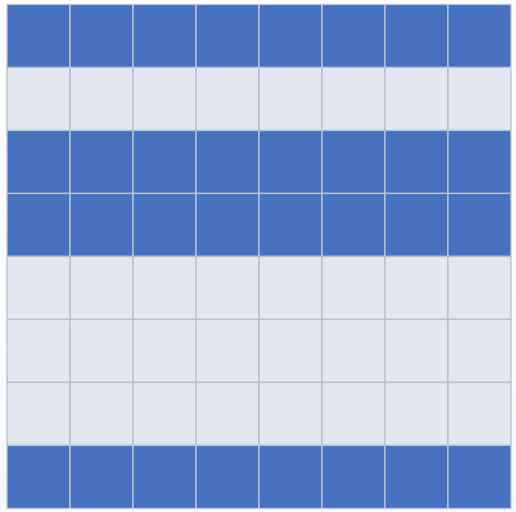
非结构化、结构化与半结构化裁剪

非结构化



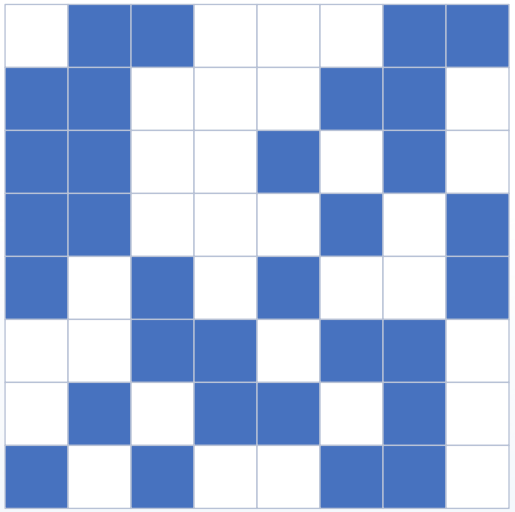
逐个权重置零；不规则，普通硬件难加速

结构化



整行（神经元）、整头或整层删除；直接加速

半结构化 2:4



每4个中留2个；GPU稀疏张量核支持

蓝色：保留　白色：置零　灰色：整行删除

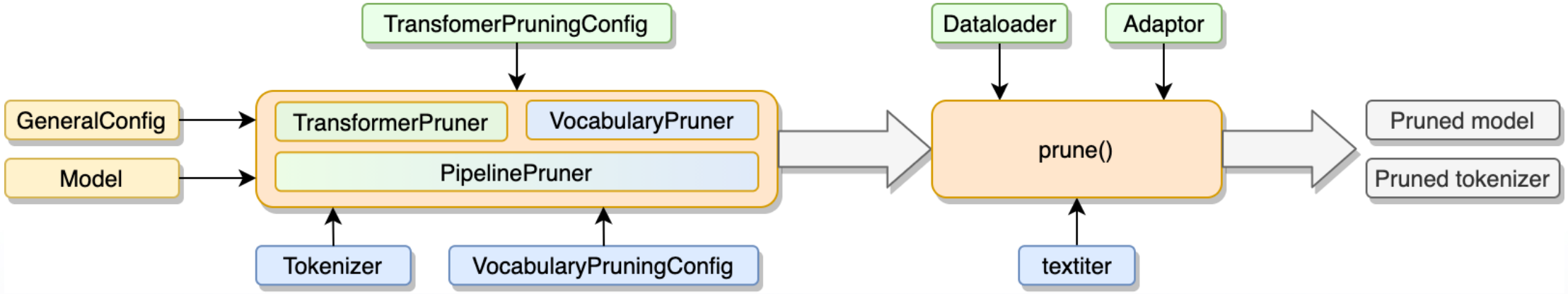
结构化裁剪：注意力头的重要性

$$I_h = \mathbb{E}_{x\sim X} \left| \partial \mathcal{L}(x) / \partial \xi_h \right|, \quad \xi_h \in \{0, 1\} \text{ 为第h个头的门控}$$

含义	I_h越大，关掉这个头对损失的影响越大，越重要
发现	测试时去掉大量注意力头，性能下降很小；有些层只留一个头即可
推广	同样思路可裁剪FFN神经元，或联合裁剪注意力头与FFN
搭配	结构化裁剪常与知识蒸馏结合，效果更好

Michel等，Are Sixteen Heads Really Better than One?，NeurIPS 2019。

TextPruner：后训练裁剪工具包



裁剪模式	做法
词表裁剪	删掉任务语料中不出现的词元，减小词向量矩阵
Transformer裁剪	找出并删掉不重要的注意力头和FFN神经元
流水线裁剪	依次进行Transformer裁剪和词表裁剪

大模型的一次性裁剪

方法	做法	效果（作者报告）
SparseGPT（2023）	逐层稀疏回归，用二阶信息补偿误差	175B模型4.5小时内裁到50%～60%
Wanda（2024）	重要性 = |W| × 输入激活范数，无须更新权重	一次前向即可完成
Minitron（2024）	结构化裁剪深度与宽度 + 蒸馏恢复	15B → 8B、4B，训练词元少至1/40

共同点：只用少量校准数据，避免重新训练整个大模型。

Frantar与Alistarh，ICML 2023；Sun等，ICLR 2024；Muralidharan等，NeurIPS 2024。

PyTorch实现：幅值裁剪与Wanda

```
W = layer.weight.data                                    # [输出，输入]
X = calib_inputs                                         # [样本数，输入]，少量校准数据
score_mag = W.abs()                                      # 幅值裁剪：只看权重
score_wanda = W.abs() * X.norm(p=2, dim=0)               # Wanda：权重 × 输入激活范数

k = W.shape[1] // 2                                      # 每个输出保留一半权重
idx = score_wanda.topk(k, dim=1, largest=False).indices
mask = torch.ones_like(W, dtype=torch.bool).scatter_(1, idx, False)
layer.weight.data = W * mask                             # 50%非结构化稀疏
```

50%稀疏（模拟）	幅值裁剪	Wanda
输出相对误差	34.0%	**9.7%**

参数量化：用更少的比特表示参数

格式	每参数比特	典型用途	7B模型权重
FP32	32	传统训练	28 GB
BF16 / FP16	16	训练与推理的默认精度	14 GB
FP8 / INT8	8	FP8训练，INT8推理（LLM.int8()）	7 GB
INT4 / MXFP4	4～4.25	权重量化推理（GPTQ、AWQ、gpt-oss）	3.5～3.7 GB
三值	1.58	从头训练的低比特模型（BitNet）	约1.4 GB

训练后量化 PTQ：直接转换已训练的模型，只需少量校准数据，最常用

量化感知训练 QAT：训练中模拟量化误差，效果更好，但需要训练数据与计算

Absmax量化：一个例子

$$X_{I8} = \text{round}\left((127 / \max|X|) \cdot X \right), \quad \text{反量化：} X \approx X_{I8} / s$$

	x_1	x_2	x_3	x_4	x_5
原始值	1.2	−0.5	4.0	−2.1	—
量化 (s = 31.75)	38	−16	127	−67	—
反量化	1.197	−0.504	4.000	−2.110	—
加入离群值60后	1.2	−0.5	4.0	−2.1	**60**
量化 (s ≈ 2.117)	3	−1	8	−4	127
反量化	**1.417**	−0.472	3.780	−1.890	60.000

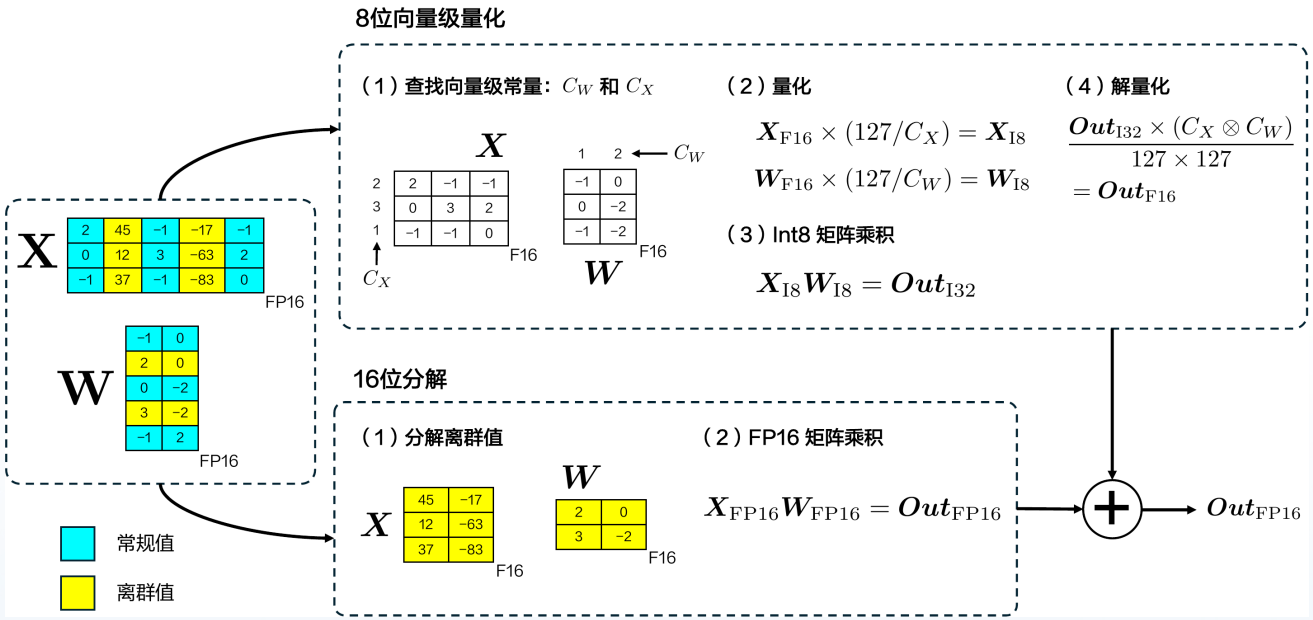
PyTorch实现：一个离群值的影响

```
def absmax_q(x, dim=None):
    m = x.abs().amax() if dim is None else x.abs().amax(dim=dim, keepdim=True)
    s = 127 / m                                          # 缩放因子
    return torch.round(x * s).clamp(-127, 127).to(torch.int8), s

W = torch.randn(4096, 4096) * 0.02
W[3, 7] = 8.0                                            # 只有一个离群值
q, s = absmax_q(W)                                       # 整个矩阵共用一个缩放因子
print((q == 0).float().mean())                           # 0.885：大多数权重变成0
q, s = absmax_q(W, dim=1)                                # 每行一个缩放因子（向量级）
print((q == 0).float().mean())                           # 0.012
```

4096×4096，一个离群值	整矩阵一个缩放因子	每行一个缩放因子
被量化为0的比例 / 平均误差	88.5% / 0.0140	**1.2% / 0.00015**

LLM.int8()：向量级量化 + 混合精度分解



$$C_{F16} \approx \Sigma_{h \in O} X^h_{F16} W^h_{F16} + S_{F16} \cdot \Sigma_{h \notin O} X^h_{I8} W^h_{I8}$$

关键数字

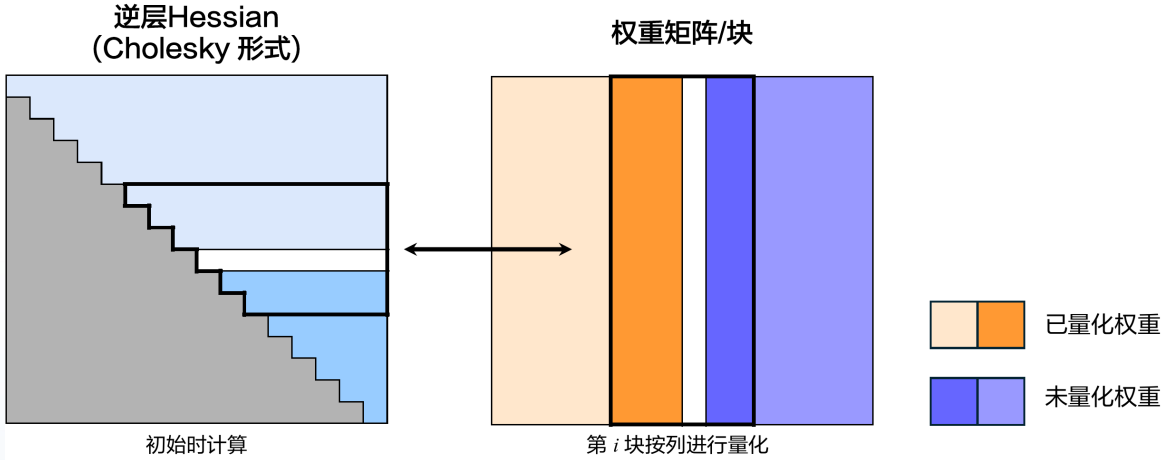
阈值α = 6.0；离群值约占0.1%；超过67亿参数时离群特征明显增多；显存约为FP16的一半

GPTQ：逐层量化并补偿误差

$$\text{argmin}_{\hat{W}} \| W X - \hat{W} X \|_2^2$$

让量化后的层输出尽量接近原输出

改进	作用
任意顺序	所有行同一列序，快得多
延迟批量更新	每次128列，提高GPU利用率
Cholesky重构	数值稳定，加少量阻尼



效果 约4个GPU小时把1750亿参数模型量化到3～4比特（作者报告）

量化的新进展

工作	思路	要点
SmoothQuant（2023）	把激活的离群幅度按通道转移到权重	权重与激活均8位
AWQ（2024）	依据激活保护约1%的显著权重	4比特权重，无需反向传播
Gemma 3 QAT（2025）	量化感知训练	27B：54GB → 14.1GB
DeepSeek-V3（2024）	FP8混合精度训练	超大规模模型的FP8训练
gpt-oss（2025）	MoE权重用MXFP4（4.25比特）	120B模型放进单张80GB显卡
BitNet b1.58（2025）	从头训练三值权重{−1, 0, 1}	20亿参数，4万亿词元

压缩方法怎么选

目标	首选方法	说明
在更小显存上部署	4比特权重量化（GPTQ、AWQ、GGUF）	无需训练，精度损失小
更小、更快的通用模型	结构化裁剪 + 蒸馏（Minitron）	需要一定训练资源
特定任务的小模型	任务蒸馏（TextBrewer）	需要教师与任务数据
极致压缩	先蒸馏或裁剪，再量化	方法可以叠加

最终都要在目标任务和目标硬件上测量：精度、延迟、显存。

本章小结

微调	以预训练参数为起点，小学习率、预热、多随机种子
参数高效精调	LoRA：$\Delta W = BA$
生成任务	标签即输入右移；提示与填充位置设为−100
知识蒸馏	软标签与温度；DistilBERT减少40%参数、保留97%效果
模型裁剪	非结构化、结构化与2:4；大模型用一次性裁剪
参数量化	离群值是难点；LLM.int8()、GPTQ、AWQ

谢 谢!

理解语言，认知社会

以中文技术，助民族复兴



长按二维码，关注哈工大SCIR

微信号：HIT_SCIR